AIPLAY / COMMUNITY STUDY GUIDE

From first song to finished scene.

Learn together: music, pictures, adapters, video and complete creative projects.

Windows, Linux and macOS companion to the editable HTML guide
25 October 2026 / source e95d81f

Install and choose a mode	2
Install on Linux	3
Open Studio on a Mac	4
Make a first YuE2 song	5
Keep and compare the song recipe	6
Build a visual world	7
Choose a MOSS recipe	8
Run one MOSS experiment	9
Try the optional Uncensored adapter	10
Train one recording region	11
Train a reviewed dataset	12
Direct one video shot	13
Use H3 references and Turbo	14
Plan a complete music video	15
Cut, finish and export	16
Short films and episodic series	17
Work with directors.md	18
Connect Codex or Claude	19
Study together and find the sources	20

Read the controls, then the readiness. The four original captures are from an empty 22 October profile. New settings captures are dated 25 October. A visible option is not proof that its engine or models are installed.

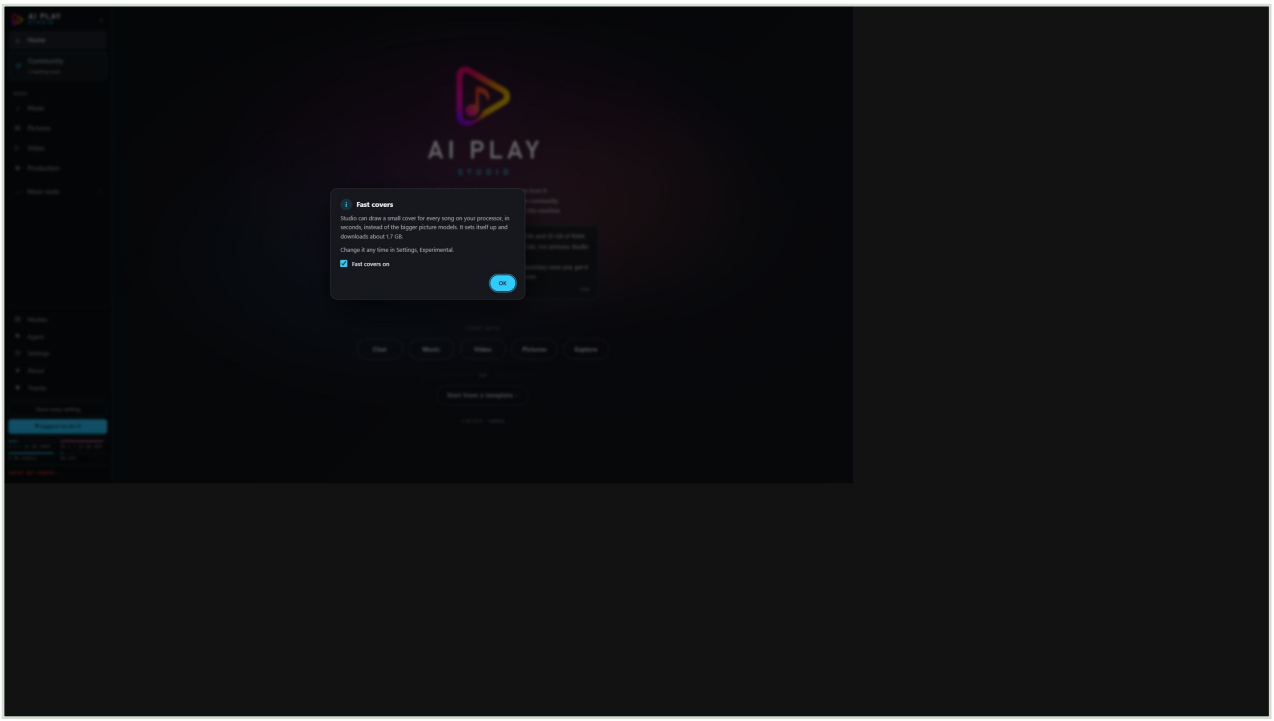
01 / SETUP

Install what you need today.

Use the official repository and the download for your platform. On Windows, get a fresh **AIPLAY Studio Setup.exe** from the Windows release. Setup 1.5 uses locked dependencies with install scripts disabled; it is still unsigned. If security software blocks it, keep protection enabled and report the exact alert.

Choose	Use it for
Music only	Native YuE2 GGUF. Automatic runtime setup is Windows x64 only; no ComfyUI or Python is required.
Full Studio	Pictures, video, music, DAW and Production. Local model features need a compatible engine and separate model files.
Use Comfy API	Optional hosted models on your own key/credits. This mode has no Music, Production or Collab screen.

Choose your mode in the launcher and keep it open. In **Models**, read the size, terms and prerequisites before Install. The app installer does not install every model or certify GPU compatibility.



Original 22 October capture: Studio home, not the launcher. The optional first-run Fast covers prompt is visible.

Source alternative: install Node.js 20+, extract the official ZIP and open a terminal in that folder. Run `npm ci --omit=dev --ignore-scripts`, then `node launcher/launcher.mjs`. This uses the launcher and its setup checks.

Windows Battery Safe: monitoring starts only after explicit On. It uses a PowerShell reader. If security software blocks it, keep protection enabled and report the exact message.

01 / SETUP, LINUX

Install on Linux.

The desktop packages target **Ubuntu 22.04/24.04 x64** and **Ubuntu 24.04 ARM64**. Choose the architecture of the computer, not its graphics card. Other distributions need their own compatibility checks.

Computer	Official download
Intel/AMD 64-bit	**AIPLAY-Studio-Linux-amd64.deb**
ARM64	**AIPLAY-Studio-Linux-arm64.deb**

1. Open the Linux release, download the matching .deb, then open it in your software installer and press **Install**. The package installer may ask for your password.
2. Open **AIPLAY Studio** from the application menu. A browser setup page downloads Studio and opens its launcher. Studio brings its own Node.js/npm; do not run the application with sudo.
3. Choose your mode. Install or connect a compatible engine, then select models separately. First launch needs internet access to GitHub and the npm registry.

A portable option

The release also has architecture-specific **.tar.gz** bundles. Extract one into a permanent folder and run **aiplay-studio**; your file manager may need **Run as a program**. It needs a supported glibc Linux system, tar, CA certificates and xdg-open. Alpine/musl is not supported by these Node binaries.

Update, repair and keep your work

Use **Update** in the launcher while Studio is stopped. For a damaged app, close Studio and run `aiplay-studio --repair`. Removing the package keeps the user app under `~/.local/share/aiplay-studio/` and the profile under `~/.aiplay-studio/`. Review them before deleting user data.

Music caveat. Native YuE2 GGUF automatic runtime installation remains Windows x64 only. On Linux, use a compatible ComfyUI music workflow or a compatible native CLI you supply. An installed Studio is not proof that a local model will run.

Download checksums and complete instructions: Linux installation guide.

01 / SETUP, MACOS

Open Studio on a Mac.

The Mac installer is a **developer preview** with a declared minimum of **macOS 13.5**. Native acceptance covers Apple silicon on macOS 14/26 and Intel on macOS 15. The minimum is not a separate full Studio test result.

About This Mac says	Choose
Apple M-series chip	**AIPLAY-Studio-macOS-arm64.dmg**
Intel processor	**AIPLAY-Studio-macOS-x64.dmg**

1. Download the matching image from the macOS preview release. Open it and drag **AIPLAY Studio** into **Applications**.
2. Eject the disk image, then open the copied **AIPLAY Studio.app** in Finder. The setup page opens in your browser, installs the app and opens its launcher.
3. Studio supplies private Node.js/npm; no separate Git or Python is needed for the app itself. Engines and models are separate installations. Reopening the running app in Finder reopens its browser page.

Security status: ad-hoc signed, without Apple Developer ID signing or notarization. macOS may block an internet-downloaded copy. Stop and report the exact message; keep protection enabled. A CI launch check does not prove Gatekeeper approval.

Where your work stays

The writable app lives under `~/Library/Application Support/AIPLAY Studio/`; the default settings and engine profile remains `~/.aiplay-studio/`. Moving the .app to the Trash keeps those user folders. Models and media stay in configured folders.

Update without a new image

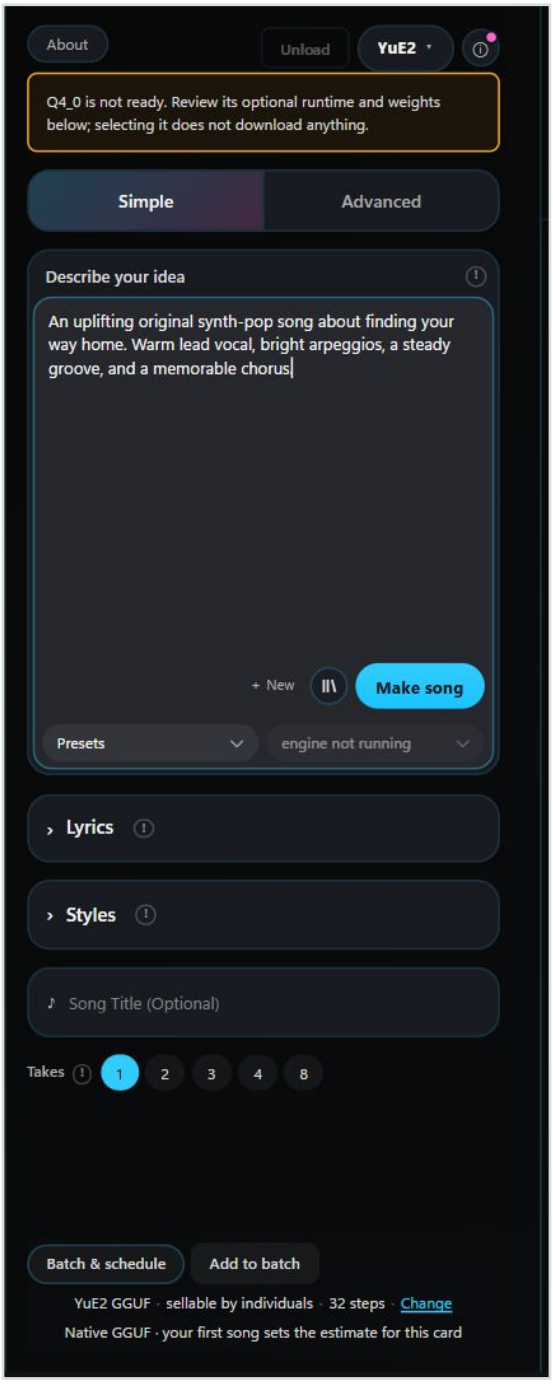
Use **Update** in the launcher while Studio is stopped. A new .dmg replaces the bootstrap/private runtime; it is not required for every app update. The linked guide includes repair and checksum instructions.

Generation is a separate check. This package does not promise local YuE2, H3 or MOSS generation on a Mac. Automatic native YuE2 GGUF runtime setup is Windows x64 only. Read the selected engine readiness report.

Read before installation: macOS installation and repair.

02 / MUSIC

Make a first YuE2 song.



Music in Simple mode: idea, presets, queued take count and an empty Library.

Open **Music**. Simple mode lets you choose **Presets...** or describe a musical idea. Review the draft, then press **Make song**. A new idea can be as simple as a mood, a story and two or three musical roles.

Try a concrete brief: warm synthpop; restrained lead vocal; rounded bass; sparse verses; a brighter layered chorus; a short instrumental bridge; a clear ending. Model adherence is something to listen for, not assume.

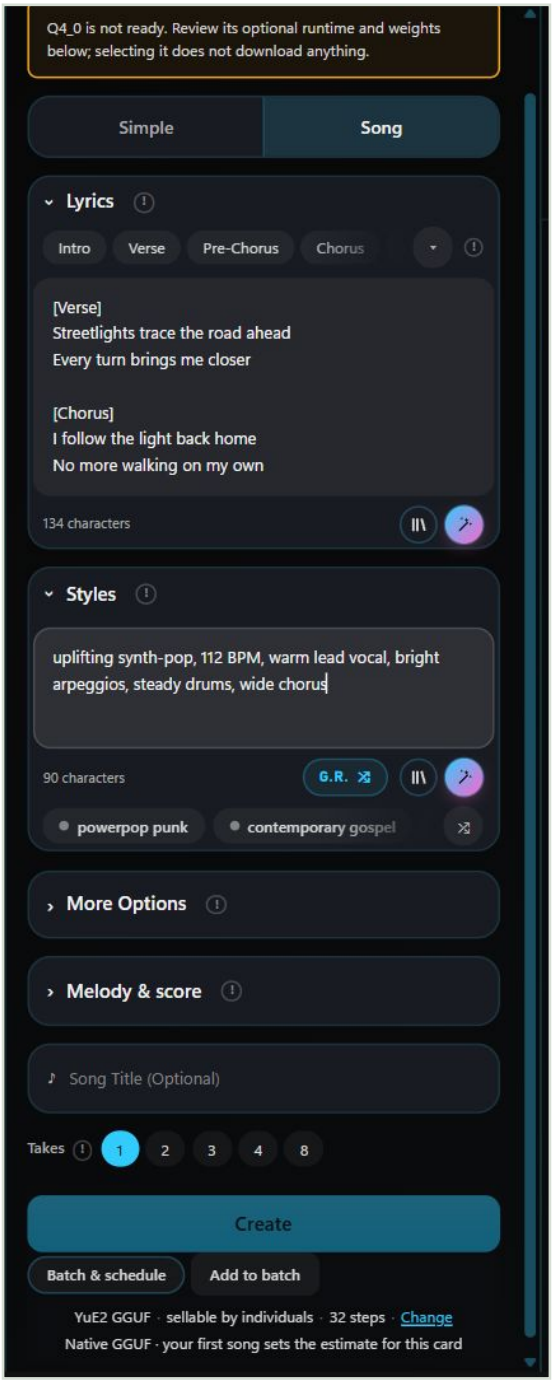
Keep the first exercise small

- Make two or three takes with the same words and style while the seed changes.
- Wait for the queued job to finish. Native output appears as a WAV in the Library.
- Listen through the ending. Note the vocal delivery, chorus, lyric intelligibility and structure.

Group prompt: which take tells the song best, and at what moment does it lose you? Share only an excerpt you have permission to share.

02 / MUSIC, CONTINUED

Keep the recipe. Compare fairly.



Direct song controls: lyrics, styles, More Options, Melody & score and Create.

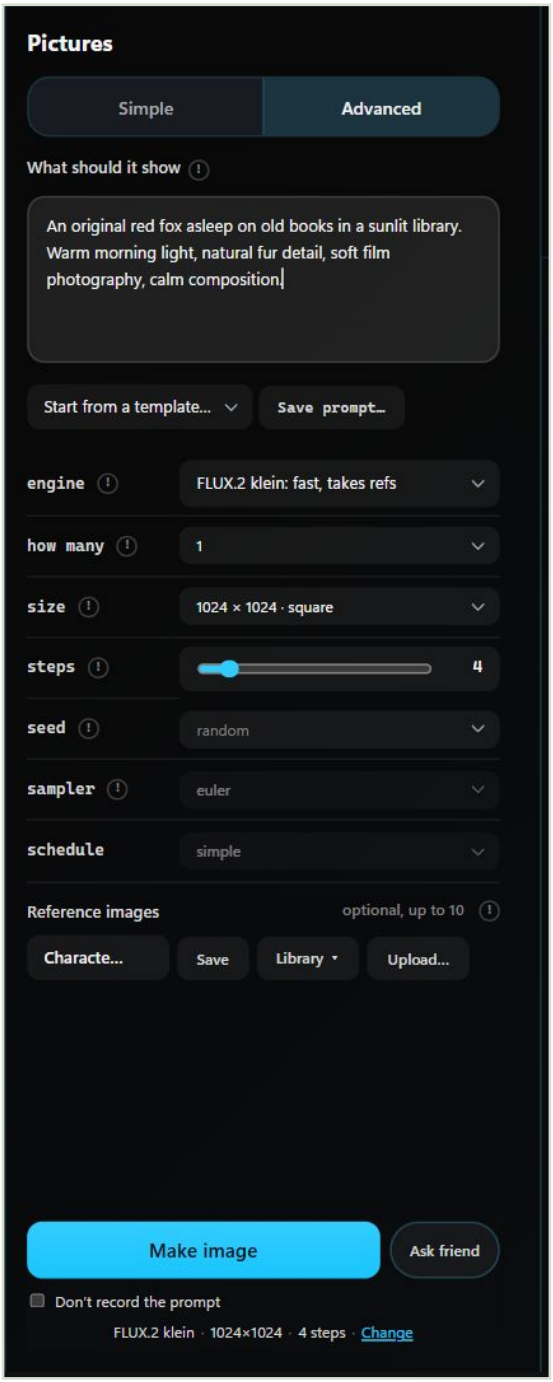
For direct lyrics and style controls, choose **Song** or **Advanced** as your build offers. The label and layout vary. Select the ready YuE2 engine before using its controls.

- Native GGUF requires nonempty style and lyrics. Put [Verse], [Chorus] and [Bridge] on their own lines.
- Keep the exact engine, model, lyrics, style, seed and settings with the chosen take. A seed is not a cross-version or cross-hardware determinism guarantee.
- Listen to the whole file. A completed job may still miss words, a section or the ending. Check ending is a prompt to listen, not a confirmed lyric diagnosis.
- Use Takes for performances of one idea. Batch & schedule handles different ideas; Studio must remain running and the computer awake.

Choose the right YuE2 path. Automatic native GGUF runtime setup is Windows x64 only. Native GGUF, Python YuE2 and YuE2 through ComfyUI have different controls and artifacts. Native GGUF has no direct singer-reference or waveform-continuation input. A new longer arrangement is a new performance.

03 / PICTURES

Build a small visual world.



Pictures in Advanced mode: prompt, engine, size, seed and Reference images.

Open **Pictures** in Full Studio. Choose an installed engine, write one clear prompt, choose a supported size and press **Make image**. Describe subject, framing, environment, light and colour.

- Start with one composition. Add exact title text later in the editor when accuracy matters.
- If the engine supports references, use Reference images > Upload... (or Use a file... in another layout). A clear single panel is often more useful than a crowded contact sheet.
- Try the subject in a second setting. Check face, silhouette, wardrobe and accessories. A reference may also carry an unwanted pose or background.
- Keep the strongest take. Use layers, selections, cutout, colour adjustments and text for ordinary finishing.

Read the selected model's terms in Models. Application, model, adapter and reference-image permissions are separate. A Fix Qwen action addresses engine support; missing model files are a separate setup check.

03 / PICTURES, MOSS CHOICES

Choose a MOSS recipe.

Qwen remains the automatic image engine and the route for references and edits. MOSS is optional text-to-image. In Pictures > Advanced choose **MOSS-Image (experimental, text only)**. Its default preset stays **Standard**; downloading a choice does not change your saved default.

Preset in Pictures	Steps / CFG	Sampler / use
Standard	25 / 2.5	ER-SDE / simple. Start here.
Spectrum H1	25 / 2.5	ER-SDE / simple. Separate acceleration source pack.
Spectrum H2	25 / 2.5	ER-SDE / simple. Another cache setting to compare.
Turbo 4	4 / 1	Euler / simple. Fast draft; needs its own adapter.
DMD 8	8 / 1	Euler / simple. Needs the MOSS DMD adapter.
Kromoss Standard	25 / 2.5	ER-SDE / simple. Separate Kromoss checkpoint.
Kromoss DMD2 8	8 / 1	Euler / simple. Kromoss plus its DMD2 adapter.
MOSS HiRes (4 MP)	25 / 4	ER-SDE / simple. Native 2048 x 2048; count 1.
NSFW Uncensored	30 / 3	ER-SDE / simple. Explicit optional adapter; strength 0.7.

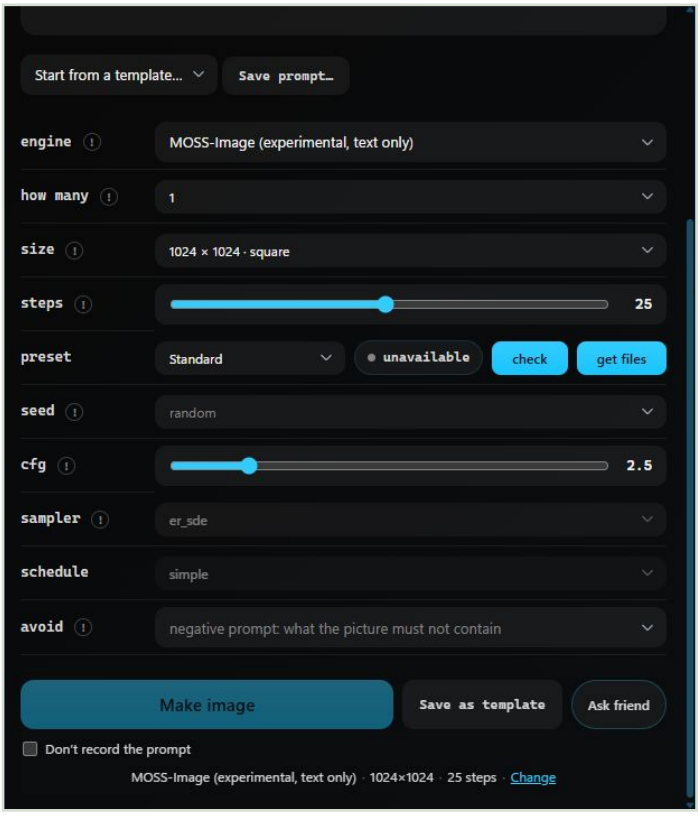
The shipped recipes use **shift 5**. Each preset names a validated combination. Do not stack arbitrary LoRAs or borrow settings from a different model. Faster recipes can change detail or composition; judge the output for your task.

Keep the route clear. MOSS currently refuses reference images, personas, editing and transparency. Use Qwen for those tasks. A community editing experiment has not become a shipped MOSS editing feature.

Recipe details and measured local comparisons: MOSS guide / Kromoss and HiRes.

03 / PICTURES, A SMALL EXPERIMENT

Run one MOSS experiment.



25 October: Standard selected, 25 steps and CFG 2.5. Models are absent, the engine is off and nothing was generated. The raw capture is presentation-cropped.

In **Models**, begin with **MOSS-Image SFT (experimental)**. Read its download size and terms before Install.

1. Open Pictures > Advanced. Select **MOSS-Image (experimental, text only)**, then a ready **Preset**. Selection alone does not install files or update ComfyUI.
2. Start with one image at 1024 x 1024. Keep prompt and seed fixed. Record preset, checkpoint, steps, CFG and time.
3. Inspect anatomy, composition, objects and text. Preserve the baseline; a faster image may be less useful.

Install the matching option

Models cards include **Kromoss INT8**, **MOSS-Image Turbo 4**, **MOSS-Image DMD 8** and **Kromoss DMD2 8**, all experimental. Kromoss DMD2 needs both the Kromoss checkpoint and its adapter.

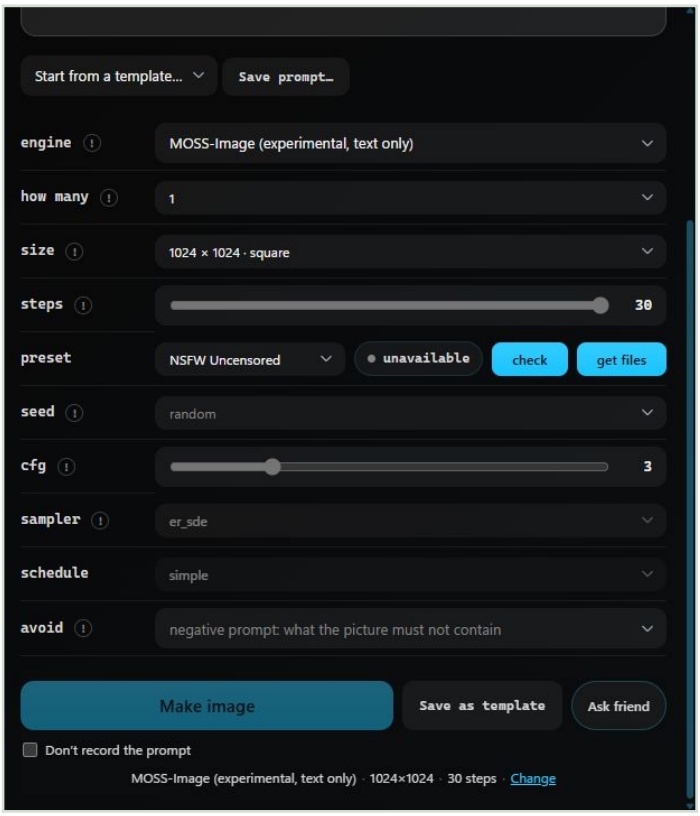
Spectrum H1/H2 has no model card. Its pinned source pack and idle-engine restart are documented in Spectrum setup. Keep a patched engine separate from an experiment.

MOSS HiRes (4 MP) uses **MOSS HiRes v0.3 (experimental)** to create a new 2048 x 2048 image, count 1. It is not an upscaler or tiled fallback. Its adapter licence is not declared; do not infer it from the base model.

03 / PICTURES, OPTIONAL ADAPTER

Try the optional adapter.

NSFW Uncensored is explicit and optional. It is not the default. Learn the controls with a neutral, fully clothed adult portrait; no explicit study-group example is needed.



25 October: Uncensored selected. Models are absent, the engine is off and nothing was generated. No adult imagery is shown. The raw capture is presentation-cropped.

Control	Recipe
Steps / CFG	**30 / 3**
Sampler	**ER-SDE / simple**
Shift	**5**
Adapter	**0.7**, model only; CLIP unchanged.

A non-explicit prompt

```
moss_xvstyle, editorial
portrait of a fully clothed
adult, navy jacket,
soft daylight, plain backdrop
```

1. In Models, read **MOSS NSFW Uncensored (optional)**, its prerequisites, size and current terms. Install only if you want this option.
2. In Pictures > Advanced choose MOSS, then **NSFW Uncensored**. Wait for readiness. Make one image and compare it with an ordinary MOSS baseline.

The trigger is **moss_xvstyle**. The adapter is about 74 MB; the publisher declares Apache 2.0. Check its card and base-model terms. Its content range and broad quality claims have not been validated here. Keep community examples non-explicit.

Preset and provenance details: MOSS image guide.

04 / TRAINING

Train one reviewed recording region.

The **Training** screen creates an experimental Audio LoRA for **YuE2 through ComfyUI**. The loop runs, but a successful checkpoint does not establish an audible improvement. Start with a listening question: which narrow change would make this useful?

Check the target before the recipe

- Use a recording you own or may train on. Choose it from From your library. Review the recording and model/tokenizer terms.
- Set Start (seconds) and an 8-180 second Region length. Use Play selected region to hear the exact material. The entire region must fit inside the recording.
- Read the readiness report: compatible ComfyUI YuE2 checkpoint, tokenizer, training/autograd support, an idle GPU and at least 10,000 MB free VRAM are required.

Run deliberately, then evaluate

- Give the adapter a name. The 24-second, rank-8, 600-step defaults are starting settings, not a proven musical recipe.
- Press Train this region only when the time is approved. It can occupy the GPU for an hour or longer; this is not a useful interruptible preview.
- Keep the run ID and use Check. When complete and checked, the adapter is saved to the LoRA shelf. This page has no intermediate checkpoint selection or resume.
- On Music, use the compatible ComfyUI engine. Make one baseline without Audio LoRA and a comparison with it. Match checkpoint, prompt, seed, duration and generation settings. Record adapter strength.

Judge with your ears. Listen to more than the training source. A loss curve or exported file does not prove voice cloning, style mastery or musical improvement. "No useful change heard" is a valid study result.

Continue reading: single-recording trainer contract and music workbench guide.

04 / TRAINING, CONTINUED

Train from a reviewed dataset.

Music Lab > Native tools offers a separate dataset trainer and optional runtime pack. It currently requires **NVIDIA CUDA with at least 11 GB VRAM**. Read the pack size and live requirements before installation.

Dataset first

1. Choose recordings and Create dataset, or import an owned local folder. Dataset creation does not certify its metadata.
2. For every song, review Style, exact Lyrics and Instrumental status, then Save song. These labels need to describe the actual recording.
3. Use Prepare recordings. Preparation converts audio; training completes the latent/code/score preparation. Editing metadata invalidates derived caches while retaining source recordings.

Choose a recipe and keep the run

4. Choose fast, balanced or thorough, then Train when ready. Optimizer, adapter and total-step overrides apply to custom.
5. Inspect Saved runs. Continue a checkpoint using a higher total step target and the same dataset fingerprint. Read status/history before retrying an uncertain start.
6. Choose Export ComfyUI LoRA, then explicitly copy/install the compatible export to the shelf. Exporting does not activate it automatically.

Apply it to the intended engine

For a combined export, select its returned filename in both **Audio LoRA** and **Planner LoRA** on **YuE2 through ComfyUI**. The ordinary native GGUF song engine does not automatically load this trained adapter. Other checkpoints are not automatically interchangeable.

A fair listening test: matched baseline + adapter take, recorded strengths, then held-out prompts or recordings. Compare both the desired change and new problems. Keep the adapter only if it helps the project.

Continue reading: native dataset training, export and limitations.

05 / VIDEO

Direct one shot before a film.

Open **Video** in Full Studio. Use an engine whose runtime, model, hardware and licence checks are satisfied. LTX offers a route for supported text/frame-guided shots; another engine may handle references and audio differently.

A useful shot brief: one person lifts a paper lantern at a harbour; medium shot; a slow push-in; the person keeps facing the camera. Give one subject, one action and one camera idea.

The pilot loop

1. Choose a supported size and a short shot length. Check what the engine actually receives: words, first/last frame, character reference, soundtrack or control video.
2. Use Render clip for one representative pilot. Review it before committing a whole scene list.
3. Watch motion with sound. Inspect first/last seconds, anatomy, object count, geography, face size and camera behavior.
4. Keep good takes. Diagnose a failure by changing one variable: action, reference, framing or seed. Record what happened.
5. Use editing or VFX for a title, grade or transition. Use new generation when the photography, subject, camera or action must change.

Match the engine to the project

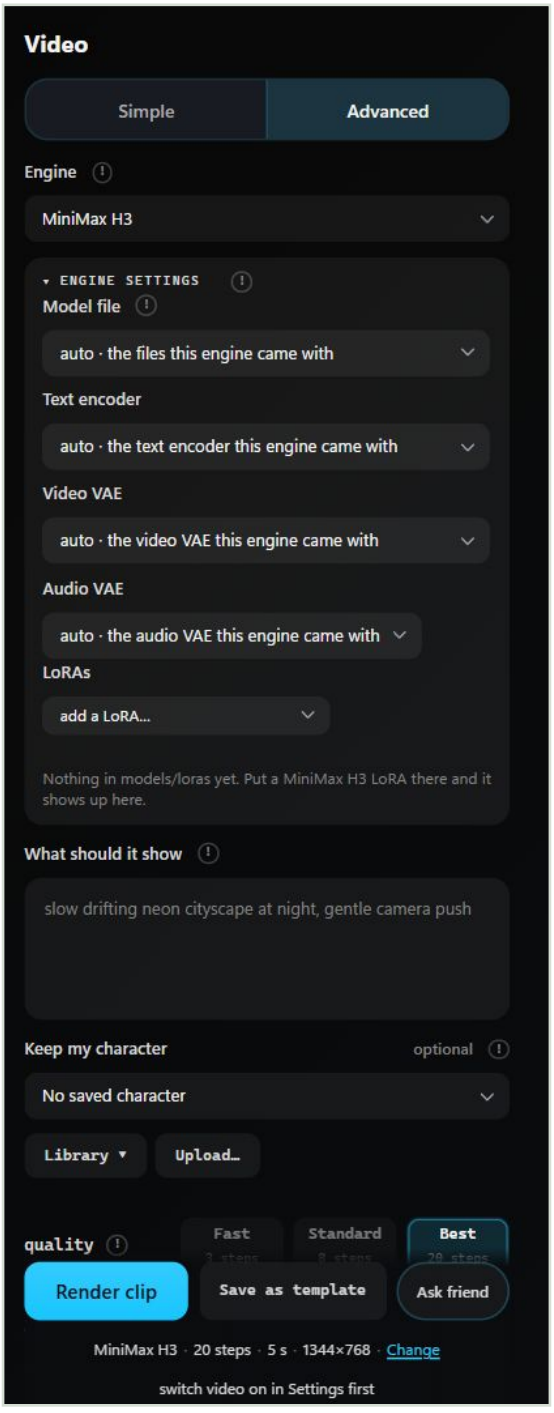
MiniMax H3 has model-specific territorial and other licence restrictions, distinct reference behavior and memory needs. Read the current Models notice for your location. Optional quantization, cache, acceleration or PTD LoRA experiments need their own integration and validation; a newer ComfyUI or PyTorch alone does not establish compatibility.

Group prompt: was a failure caused by the shot design, reference, seed or an unresolved engine limit? It is fine to keep the cause uncertain until a controlled comparison answers it.

Continue reading: directing notes and MCP video workflows.

05 / VIDEO, H3 REFERENCES

Use H3 references and Turbo.



25 October: the empty setup shows Best / 20 steps. Models are absent and video is off. This is not an installed 8-step reference recipe or a completed render.

In Models, review **Video references - MiniMax H3 ref2va** and its territorial/other licence conditions. Then open **Video > Advanced > MiniMax H3**.

Control	Follow-along setting
Components	Model file, Text encoder, Video VAE and Audio VAE: **auto**.
Reference	References > **Add a picture...**. Name its role: **<Picture 1> is Mira**.
REF2V recipe	**8 steps**; **res_multistep / simple**; shifts **12 / 3**; adapter strength **1**.
Turbo	V1.1 preferred when installed; V1.0 fallback. Do not add a second manual Turbo LoRA.

Keep component selectors on **auto** for this path. The displayed Standard steps are dynamic: verify **8** when the reference files are installed. Four-step references use a separate v0.1 route. Plain/Quality paths are separate too.

The measured comparison: one matched Singularity V2 adapter pair on RTX 4070 Ti SUPER 16 GB, 1344 x 768, 124 frames at 24 fps and 8 steps ran in **260.611 s (V1.0)** versus **246.861 s (V1.1)**, about 5.3% less time. V1.1 changed more of the reference scene. This is not a general speed promise or overall quality win.

Singularity V2 was the test checkpoint, not a new automatic checkpoint default. A still cannot establish motion, lip sync or vocal fit; watch the whole pilot. See H3 Turbo details.

06 / PRODUCTION

Plan the whole music video.

Keep the accepted song, a clear brief, reusable references, one board per scene and chosen takes. **Production** holds the plan; **Studio** holds the cut. A complete project is more than a folder of clips.

Song, brief and direction

1. **Upload & analyze.** Create a project with New video..., attach the accepted song and inspect the beat grid. Listen to the pulse: half-time music can be detected at double tempo.
2. **Creative interview.** Decide medium, tone, narrative approach, aspect ratio, supported size and constraints before rendering assets.
3. **Script & direction.** Create the story, style bible, named characters/props/locations and a storyboard per scene. The current UI needs an outside MCP assistant for the first whole bible, using mv_bible_spec and mv_set_bible. Existing bible parts are editable in Production; built-in Chat does not expose that whole-document action.
4. **Story review.** Run the free review/lint checks. Resolve absent boards and undeclared references. Without a board, a scene can fall back to a generic performance shot.

References, boards and clips

5. **Characters & props > Backgrounds > Storyboards.** Generate or import references, pick takes and review all boards together. Check cast, wardrobe and geography. A chosen reference change can mark dependent work stale.
6. **Video clips.** Make a pilot first. Once accepted, render the approved scenes. Keep earlier takes and choose among them. Read stale findings before regenerating.

Ask an assistant: Read directors.md and the live pipeline guide. Read the existing project, write the proposed brief and scene plan, run the free review checks, then present one concrete pilot. Do not download or render yet.

06 / PRODUCTION, CONTINUED

Cut, finish and export.

Build the cut

Use **Move to Studio** to assemble selected scene takes with the song. Edit rhythm, trim pauses and inspect joins. Preserve a separate edit before **Rebuild the Studio timeline**: rebuilding replaces that project's composed timeline with current picks.

Use titles, credits, grades, transitions and sound work for finishing. New generation is usually the wrong tool for changing a title or repairing a cut.

Choose an export route that preserves the work

Where available, **Render the finished video** uses ffmpeg to compose from source media. It currently covers source video clips, **one audio item** and its own optional fade, beat-pulse and grade settings. It is **not a full replay of Studio's layers, effects or multiple-track audio**.

Check titles, effects and additional audio before choosing that route. Bake finishing into source media or use a verified export route. Studio's **Export video** captures the browser canvas in real time and has different capture constraints. Building a project or timeline does not itself create a finished movie file.

Review the delivered file

- Play the actual export from beginning to end, not only its timeline preview.
- Check sound, picture, titles, credits, framing, joins, pacing and ending.
- Collect timecoded notes and repair the smallest affected set of shots. Keep the project, chosen sources and receipts with the delivered version.

Group exercise: bring a short public-safe cut and one timecoded question. "The cut at 0:18 interrupts the phrase" gives the group something concrete to improve.

Source: current Production cut/export interface and offline renderer scope.

07 / LONGER STORIES

Short films and episodic series.

A short film

Start with a logline, a beginning/turn/ending and a small scene list. Separate picture, dialogue/narration, music and effects. Keep each shot within the selected engine's supported workflow and assemble the longer story editorially.

Production video projects can hold the scene plan, with external planning/MCP for the first bible. Handle narration and sound as explicit assets. A text brief alone does not create a finished synchronized film.

A series

Create **one project per episode**. Maintain a shared series bible: stable names, wardrobe, locations, props, palette and story facts. Record what changes in each episode and the final state the next episode inherits.

Carry the actual approved reference images into each project, for example with **mv_import_asset**. Projects do not automatically share one cast store. Matching prose alone does not preserve a face.

A repeatable production loop

1. Approve the episode outline and continuity changes before asset work.

2. Prove one representative dialogue, action or performance scene as the pilot.

3. Record accepted takes and preserve the exact chosen references.

4. Build the cut, layer its sound and review across scene boundaries.

5. Export a review version and repair the smallest affected shot set.

For a narration-led project, Production also offers an audiobook workflow: ingest/select chapters, plan bundles, audition and assign voices, narrate, add cues/beds and mix. Its own setup and source permissions apply; visual editing remains a separate task.

Group exercise: plan a two-scene short or a second episode using the same chosen character image. Share the continuity checklist before rendering.

08 / WORKING MEMORY

Give the assistant directors.md.

Use the supplied **directors.md** template beside your project planning files. Replace its bracketed fields. It holds the creative brief, continuity bible, shot contract, approved scope, comparison recipes and handoff log.

This is a suggested project practice, not a Studio configuration format. AIPLAY does not automatically import it as a production bible. Tell your assistant to read it; add that instruction to **AGENTS.md** for Codex or **CLAUDE.md** for Claude Code.

```
Before working on this creative project, read directors.md.
Read the current Studio project and live tool schemas.
Honor the approved scope and keep the handoff log current.
Report only the renders, listening checks and exports that happened.
```

Write decisions and evidence

- Record the exact chosen reference, scene ID, take, engine and reason.
- Separate an intention from an observation: “slow push-in requested” versus “we watched the camera move correctly.”
- Keep approved GPU, download, training, paid-service and sharing scope explicit. Honor authorization already given; present a concrete proposal for work outside it.
- After an uncertain start, read saved status/history before retrying. Do not duplicate expensive work by guessing.

Make the next handoff easy

Leave a short latest entry: accepted constraints, current project/run IDs, what changed, what was actually watched or heard, unresolved issues and the next useful action. Carry exact reference files between episodes.

Group exercise: let a fresh assistant resume from this file and saved Studio state. Note what it could not recover, then improve the handoff.

09 / MCP

Connect Codex or Claude.

Start Studio, then open **Agent > MCP server** and copy its generated configuration. It includes the **actual Node executable and absolute MCP script path** for your installation. This matters for Linux/macOS bundles: private Node may not be on the assistant PATH. The bridge talks to the running local Studio API.

Codex - when Node is on PATH

```
codex mcp add aiplay -- node "C:/YOUR/AIPLAY-Studio/server/mcp.js"
codex mcp list
```

These are Windows path examples. Replace the placeholder, or use the generated command and arguments from Agent. The HTML guide also includes a TOML example. In the terminal client, /mcp shows active servers.

Claude Code - when Node is on PATH

```
claude mcp add --transport stdio aiplay -- node "C:/YOUR/AIPLAY-Studio/server/mcp.js"
claude mcp get aiplay
claude mcp list
```

Use /mcp inside Claude Code to inspect status. Claude Desktop chat has separate configuration; use Studio's generated JSON and that client's current instructions.

Verify with a free read

Ask for **studio_capabilities** with **showcase:false**, **studio_screen_info** for the intended screen and **pipeline_guide** for the task. Read the current schemas before project or training actions.

First request: Read capabilities and Music readiness. Explain installed engines and missing requirements. Do not install, download, train, render, delete or publish anything.

If connection fails: check the Studio address and exact Node/script paths generated by Agent. Set AIPLAY_URL if Studio uses another port; leave it local. MCP is separate from adding an API key for built-in Chat. Your outside assistant still has its own account and data policy; material returned to it may reach its provider.

Official references: OpenAI MCP and Claude Code MCP. Client examples retained from the 22 October guide; check current client instructions.

10 / STUDY GROUP

Share the useful experiment.

Bring one clear question to each session. A reproducible failure can teach more than a polished clip with no context.

- **Show the task.** Give the Studio build line, engine/model, main settings and the one change tested.
- **Show evidence.** Use an owned excerpt, public-safe screenshot or real playback. Separate an interpretation from what you watched or heard.
- **Compare fairly.** Match relevant settings, compare audio at similar loudness and record adapter/reference strengths.
- **Protect private material.** Crop keys, paths, names, recent projects and unrelated media. Obtain permission before sharing another person's face, voice or work.
- **Ask a focused question.** A timecoded lyric, cut or continuity issue is easier to discuss than "Is it good?"
- **Keep originals.** Preserve baseline, chosen takes and projects. Review the actual exported file and required permissions/credits before release.

Suggested session sequence

Setup > YuE2 songs > pictures/references > adapter comparison > one video pilot > bible/boards > cut/export > film or series handoff. The accompanying **discord-posts.html** contains eight short invitations ready to adapt.

Reference shelf

AIPLAY: repository / installer release
Installation / Linux / macOS
MOSS presets / H3 Turbo
native YuE2 controls and limits
workbench and native training / single-recording training
directing notes / MCP workflow map
Production UI / offline renderer scope
OpenAI AGENTS.md / Claude project memory

This edition follows source e95d81f and the inspected 25 October 2026 controls. Linked main-branch files can change. Use the current build's readiness and model terms for the feature you choose.